

模拟调制实验

2026 年 5 月 22 日

班级：2024217802 姓名：严子竣 学号：2024212622

1 高斯噪声的产生与绘制

1.1 AWGN 的产生与绘制

本部分使用低通滤波器从白噪声产生带限高斯噪声，并绘制窄带高斯噪声及其同相分量波形。参数设置中带宽 $B = 100$ Hz，中心频率 $f_c = 2000$ Hz，采样率远高于载频以满足奈奎斯特条件。

代码如下（对应文件 AWGN.m）：

```
1 clear; clc; close all;
2
3 %% 参数设置
4 B = 100; % 带宽 (Hz)
5 fc = 2000; % 中心频率 (Hz)，远大于 B
6 fs = 25 * fc; % 采样率 (Hz)，满足 fs >= 2(f_c+B)
7 T = 0.01; % 信号时长 (s)
8 N = round(T * fs); % 样本点数
9 t = (0:N-1)/fs;
10
11 % 噪声功率谱密度 (假设单位: V^2/Hz)
12 N0 = 1; % 单边带功率谱密度
13 sigma2 = N0 * B; % 噪声方差
14
15 % 低通滤波器 (截止频率 B/2)
16 lpFilt = designfilt('lowpassfir', 'PassbandFrequency', B/2, ...
17 'StopbandFrequency', B/2 + B/5, 'SampleRate', fs);
18
19 % 生成独立同相/正交高斯白噪声
20 n_I = filter(lpFilt, sqrt(sigma2) * randn(N, 1));
21 n_Q = filter(lpFilt, sqrt(sigma2) * randn(N, 1));
22
23 n = n_I .* cos(2*pi*fc*t) - n_Q .* sin(2*pi*fc*t);
24
25 figure('Position', [100, 100, 1200, 600]);
26
27 subplot(2,1,1);
28 plot(t, n, 'k', 'LineWidth', 0.8);
29 xlabel('时间 t (s)'); ylabel('幅度'); title('实窄带高斯噪声 n(t)');
```

```

30 grid on; xlim([0, T]);
31
32 subplot(2,1,2);
33 plot(t, n_I, 'b', 'LineWidth', 1);
34 xlabel('时间 t(s)'); ylabel('幅度'); title('同相分量 n_I(t)');
35 grid on; xlim([0, T]);

```

仿真结果:

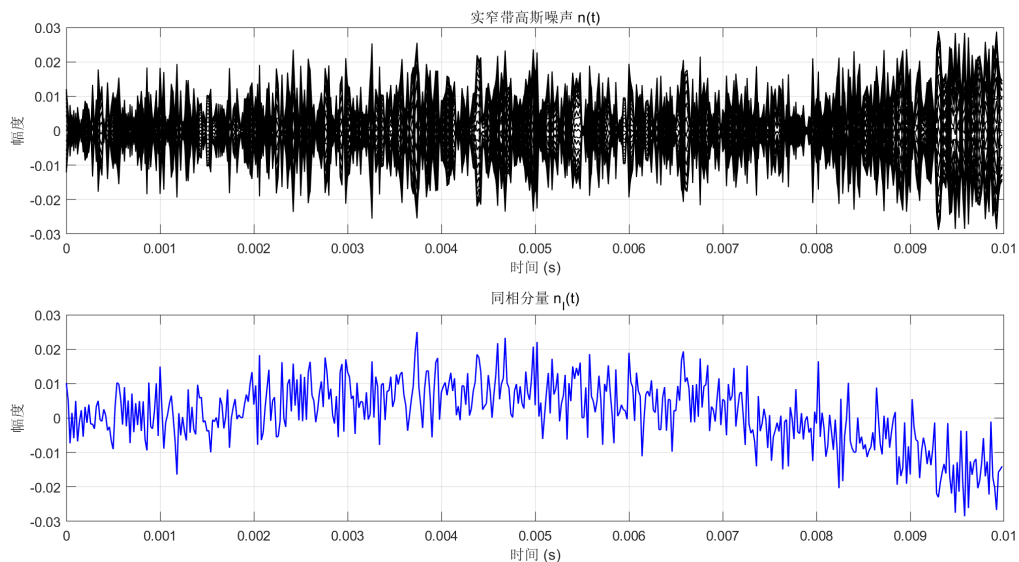


图 1: 加性白高斯噪声及其同相分量

1.2 窄带白高斯噪声的分布与自相关验证

分别考虑采样率等于带宽和大于带宽两种情况, 验证同相分量的高斯分布和自相关特性。对于 $f_s = B$, 采样点独立; 对于 $f_s > B$, 通过低通滤波产生相关序列。直方图与理论高斯曲线吻合, 自相关函数在 $f_s = B$ 时近似冲激, 在 $f_s > B$ 时符合 $\text{sinc}(B\tau)$ 函数。

代码如下 (对应文件 Gauss_noise.m):

```

1 clear; clc; close all;
2
3 %% 参数设置
4 B = 100;           % 带宽 (Hz)
5 fc = 2000;         % 中心频率 (Hz), 远大于 B
6 fs1 = B; %基础要求
7 fs2 = 4 * B; %进阶要求
8 T = 5;             % 信号时长 (s)
9
10 N1 = round(T * fs1); % 样本点数
11 N2 = round(T * fs2);
12
13 t1 = (0:N1-1)/fs1;
14 t2 = (0:N2-1)/fs2;
15

```

```

16 % 噪声功率谱密度 (假设单位: V2/Hz)
17 N0 = 1; % 单边带功率谱密度
18 sigma2 = N0 * B; % 噪声方差
19
20 n_I_independent = sqrt(sigma2) * randn(N1, 1); % 独立高斯序列
21
22 figure('Position', [100, 100, 1200, 800]);
23 % 分布验证
24 subplot(2,2,1);
25 histogram(n_I_independent, 'Normalization', 'pdf', 'BinWidth', 0.2);
26 hold on;
27 x = linspace(-4*sqrt(sigma2), 4*sqrt(sigma2), 200);
28 pdf_theory = (1/sqrt(2*pi*sigma2)) * exp(-x.^2/(2*sigma2));
29 plot(x, pdf_theory, 'r-', 'LineWidth', 2);
30 xlabel('n_I'); ylabel('概率密度'); title('fs=B: 同相分量直方图');
31 legend('仿真', '理论高斯'); grid on;
32
33 % 自相关验证
34 subplot(2,2,2);
35 max_lag = round(0.1*fs1); % 观察0.1秒内
36 [R, lags] = xcorr(n_I_independent, max_lag, 'normalized');
37 plot(lags/fs1, R, 'b-');
38 hold on;
39 stem(0, 1, 'r', 'LineWidth', 2, 'MarkerSize', 8);
40 xlabel('时延(s)'); ylabel('归一化自相关'); title('fs=B: 自相关函数');
41 xlim([-0.1, 0.1]); grid on;
42 legend('仿真', '理论冲激');
43 % 正态性检验
44 [h1, p1] = jbtest(n_I_independent);
45 fprintf('fs=B: Jarque-Bera 检验 h=%d, p=%0.4f\n', h1, p1);
46 fprintf('实测方差: %0.4f, 理论方差: %0.4f\n', var(n_I_independent), sigma2);
47
48 % 低通滤波器 (截止频率 B/2)
49 lpFilt = designfilt('lowpassfir', 'PassbandFrequency', B/2, ...
50     'StopbandFrequency', B/2 + B/5, 'SampleRate', fs2);
51
52 sigma2_in = sigma2 * fs2 / B; % 输入白噪声方差
53 white_noise = sqrt(sigma2_in) * randn(N2, 1);
54 n_I_correlated = filter(lpFilt, white_noise);
55
56 % 去除滤波器暂态 (取后半部分)
57 filter_order = length(lpFilt.Coefficients);
58 n_I_correlated = n_I_correlated(filter_order:end);
59 N2_eff = length(n_I_correlated);
60 t2_eff = (0:N2_eff-1)/fs2;
61
62 % 分布验证
63 subplot(2,2,3);
64 histogram(n_I_correlated, 'Normalization', 'pdf', 'BinWidth', 0.2);
65 hold on;

```

```

66 x2 = linspace(-4*sqrt(sigma2), 4*sqrt(sigma2), 200);
67 pdf_theory2 = (1/sqrt(2*pi*sigma2)) * exp(-x2.^2/(2*sigma2));
68 plot(x2, pdf_theory2, 'r-', 'LineWidth', 2);
69 xlabel('n_I'); ylabel('概率密度'); title('fs>B:同相分量直方图');
70 legend('仿真', '理论高斯'); grid on;
71
72 %自相关验证
73 subplot(2,2,4);
74 max_lag_sec = 0.05; % 观察 50 ms
75 max_lag_samp = round(max_lag_sec * fs2);
76 [R_corr, lags_corr] = xcorr(n_I_correlated, max_lag_samp, 'normalized');
77 lags_time = lags_corr / fs2;
78 % 理论自相关 (归一化): R_theory(tau) = sinc(B * tau)
79 tau_theory = linspace(-max_lag_sec, max_lag_sec, 500);
80 R_theory = sinc(B * tau_theory);
81
82 plot(lags_time, R_corr, 'b-', 'LineWidth', 1.5); hold on;
83 plot(tau_theory, R_theory, 'r--', 'LineWidth', 1.5);
84 xlabel('时延\tau(s)'); ylabel('归一化自相关'); title('fs>B:自相关函数');
85 legend('仿真', '理论\sinc(B\tau)'); grid on;
86 xlim([-max_lag_sec, max_lag_sec]);
87 % 正态性检验
88 [h2, p2] = jbtest(n_I_correlated);
89 fprintf('fs>B:Jarque-Bera检验 h=%d, p=%.4f\n', h2, p2);
90 fprintf('实测方差: %.4f, 理论方差: %.4f\n', var(n_I_correlated), sigma2);

```

命令行输出:

```

1 fs = B: Jarque-Bera 检验 h = 0, p = 0.5000
2 实测方差: 100.8026, 理论方差: 100.0000
3 fs > B: Jarque-Bera 检验 h = 0, p = 0.0888
4 实测方差: 110.7303, 理论方差: 100.0000

```

仿真结果:

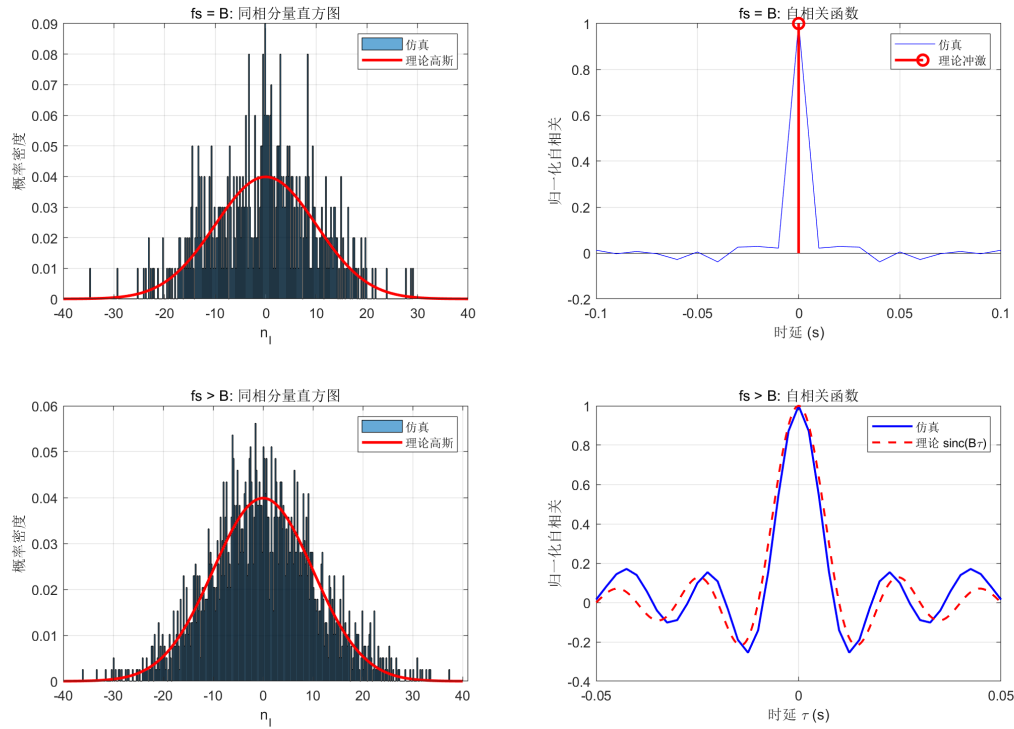


图 2: 分布验证与自相关验证

2 矩形基带信号

生成长度为 $L = 10$ 的随机比特，映射为 ± 1 符号，每个符号持续 $T_s = 2/B$ 秒，通过重复每个符号得到矩形波形。采样率设置为每个符号 100 个样点。

代码如下（对应文件 rect.m）：

```

1 clear; clc; close all;
2
3 %% 参数设置
4 B = 100;           % 带宽 (Hz)
5 Ts = 2/B;         % 符号周期
6 samples_per_symbol = 100;
7 fs = samples_per_symbol/Ts; % 采样频率
8 L = 10;           % 矩形个数
9
10 %a_n的产生
11 bits = randi([0, 1], 1, L); % 长度为 L 的随机比特
12 symbols = 2 * bits - 1;
13
14 %矩形波的产生
15 m = repelem(symbols, samples_per_symbol);
16
17 t_total = L * Ts; % 总时长 (s)
18 t = linspace(0, t_total, length(m)); % 时间向量
19

```

```

20 figure;
21 plot(t, m, 'b-', 'LineWidth', 1.5);
22 xlabel('时间 (s)');
23 ylabel('幅度');
24 title('矩形基带信号 m(t)');
25 grid on;
26 ylim([-1.2, 1.2]);

```

仿真结果:

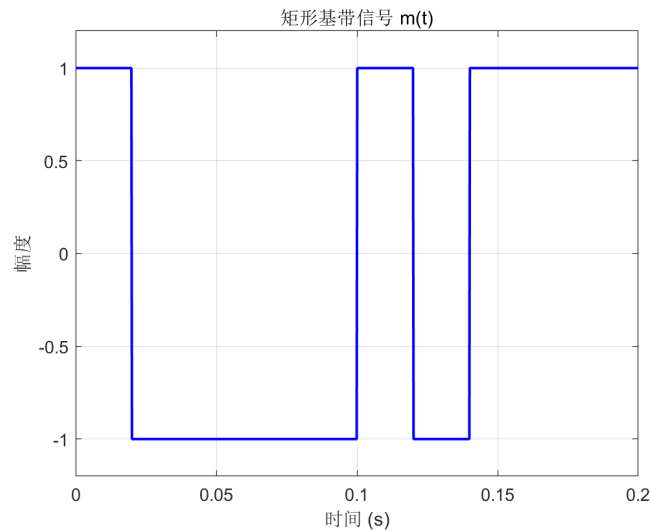


图 3: 矩形基带信号

3 DSB 调制解调

生成窄带高斯噪声和矩形基带信号，进行 DSB 调制（与载波相乘），加入噪声后进行相干解调（再次与载波相乘后低通滤波），得到解调输出 $m(t) + n_c(t)$ 。绘制窄带噪声同相分量、基带信号、已调信号和解调输出波形。

代码如下（对应文件 DSB.m）:

```

1  %参数设置
2  B = 100;          % 基波带宽(Hz)
3  fc = 1000;        % 载波频率(Hz)
4  samples_per_symbol = 2000;
5  fs = samples_per_symbol/Ts;          % 采样频率
6  L = 10;
7
8  % 时间轴
9  t_total = L * Ts;          % 总时长
10 t = 0:1/fs:t_total-1/fs;    % 时间向量
11 Nsamples = length(t);      % 总采样点数
12
13 % 设定低通滤波器参数
14 f_cut = B/2;              % 低通滤波器截止频率 (Hz)
15 % 使用 fir1 设计等波纹 FIR 滤波器

```

```

16 filter_order = 200;           % 滤波器阶数
17 h_lp = fir1(filter_order, f_cut/(fs/2), 'low'); % 归一化截止频率
18
19 %噪声产生
20 N0 = 1e-5;
21 Pn = N0 * B;
22 white_noise = randn(1, Nsamples);
23 n_c_filt = filter(h_lp, 1, white_noise);
24 %计算实际功率, 并调整到 Pn
25 actual_power = var(n_c_filt);
26 n_c = n_c_filt * sqrt(Pn / actual_power);
27 %正交分量
28 white_noise_q = randn(1, Nsamples);
29 n_s_filt = filter(h_lp, 1, white_noise_q);
30 actual_power_q = var(n_s_filt);
31 n_s = n_s_filt * sqrt(Pn / actual_power_q);
32
33 %矩形波
34 bits = randi([0, 1], 1, L); % 长度为 L 的随机比特
35 symbols = 2 * bits - 1;
36 m = repelem(symbols, samples_per_symbol);
37
38 %载波
39 carrier_cos = cos(2*pi*fc*t);
40 carrier_sin = sin(2*pi*fc*t);
41 s_t = m .* carrier_cos; % DSB调制
42 n_t = n_c .* carrier_cos - n_s .* carrier_sin; % 窄带噪声
43 r_t = s_t + n_t; % 接收信号
44
45 %相干解调
46 demod = 2 * r_t .* carrier_cos;
47 n_t = n_c .* carrier_cos - n_s .* carrier_sin;
48 demod_out = filtfilt(h_lp, 1, demod);
49 % 由于滤波初始瞬态, 丢弃前 filter_order 个样本
50 start_idx = filter_order + 1;
51 demod_out = demod_out(start_idx:end);
52 t_adj = t(start_idx:end);
53 m_adj = m(start_idx:end);
54 n_c_adj = n_c(start_idx:end);
55
56 %绘制波形
57 figure('Name', 'DSB调制与解调波形', 'Position', [100, 100, 1200, 800]);
58
59 % 子图1: 窄带高斯噪声的同相分量 n_c(t) (局部, 显示前 0.03 秒)
60 t_limit_nc = 0.03; % 显示时间长度
61 idx_nc = t_adj <= t_limit_nc;
62 subplot(2,2,1);
63 plot(t_adj(idx_nc), n_c_adj(idx_nc), 'b', 'LineWidth', 1);
64 xlabel('时间_t(s)'); ylabel('幅度');
65 title('窄带高斯噪声同相分量_n_c(t)');

```

```

66 grid on;
67
68 % 子图2: DSB 同相分量 = 基带信号 m(t)
69 subplot(2,2,2);
70 plot(t, m, 'b-', 'LineWidth', 1.5);
71 xlabel('时间/(s)');
72 ylabel('幅度');
73 title('矩形基带信号 m(t)');
74 grid on;
75 ylim([-1.2, 1.2]);
76
77 % 子图3: DSB 已调信号 s(t)
78 idx_s = t <= Ts;
79 subplot(2,2,3);
80 plot(t(idx_s), s_t(idx_s), 'g', 'LineWidth', 1);
81 xlabel('时间/(s)'); ylabel('幅度');
82 title('DSB 已调信号 s(t)');
83 grid on;
84
85 % 子图4: DSB 相干解调输出
86 idx_full = t_adj <= t_total;
87
88 subplot(2,2,4);
89 plot(t_adj(idx_full), m_adj(idx_full), 'r--', 'LineWidth', 1.2); hold on;
90 plot(t_adj(idx_full), demod_out(idx_full), 'b-', 'LineWidth', 1);
91 xlabel('时间/(s)'); ylabel('幅度');
92 title('DSB 相干解调输出');
93 legend('原始 m(t)', '解调输出 m(t)+n_c(t)', 'Location', 'best');
94 grid on;
95 ylim([-2, 2]);
96 hold off;
97
98 sgtitle('DSB 调制与解调实验波形');

```

仿真结果:

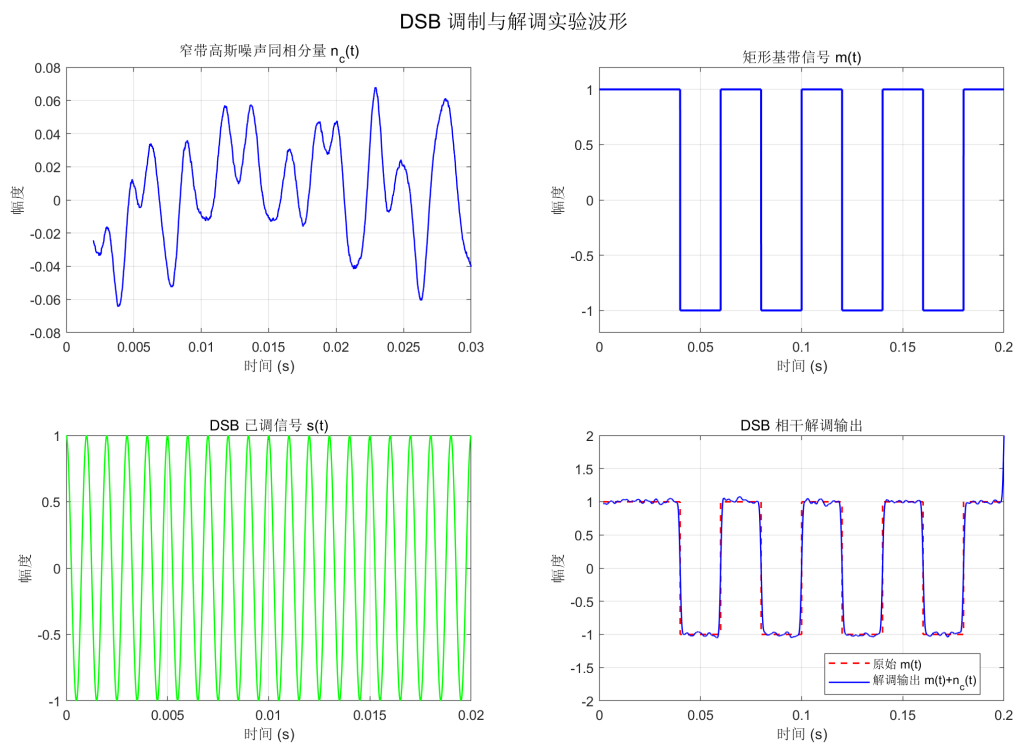


图 4: DSB 调制解调 (1)

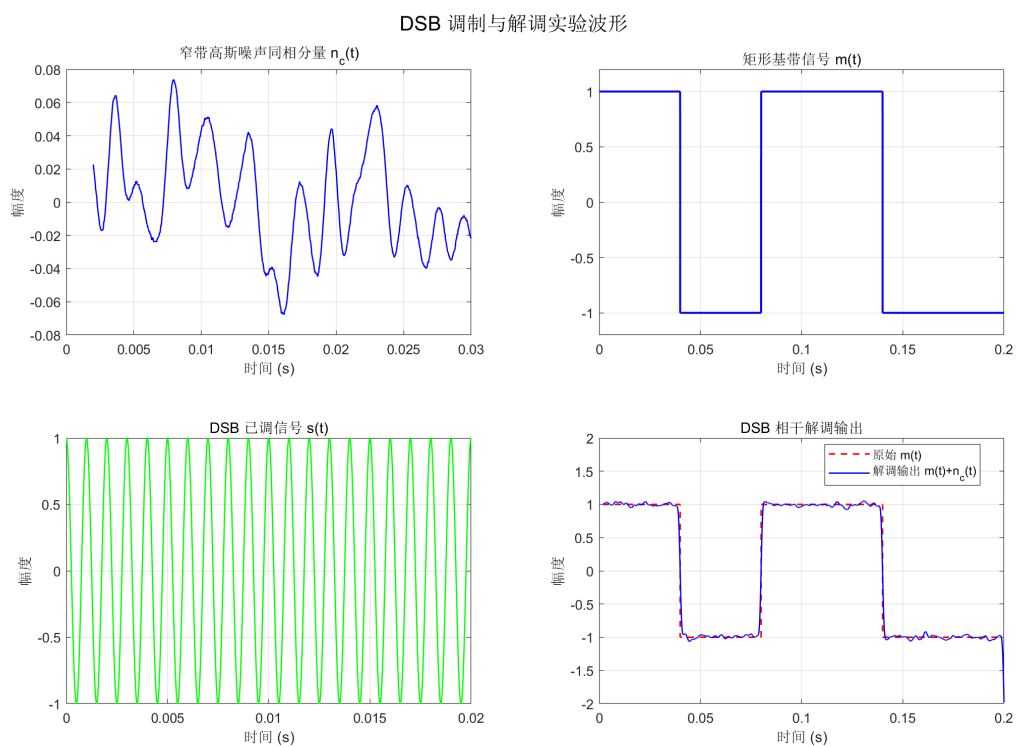


图 5: DSB 调制解调 (2)

4 AM 调制解调

AM 调制信号为 $s(t) = [1 + am(t)] \cos 2\pi f_c t$ ，其中调制指数 $a = 0.8$ 。相干解调后需去除直流分量得到 $am(t) + n_c(t)$ 。绘制窄带噪声同相分量、基带信号、AM 同向分量、已调信号和解调输出波形。

代码如下（对应文件 AM.m）：

```

1 clear; clc; close all;
2
3 %% 参数设置
4 B = 100; % 基带信号带宽 (Hz)
5 fc = 1000; % 载波频率 (Hz)
6 Ts = 1 / B; % 符号周期 (s)
7 samples_per_symbol = 2000;
8 fs = samples_per_symbol / Ts; % 采样频率 (Hz)
9 L = 10; % 符号个数
10
11 t_total = L * Ts; % 总时长 = 0.1 s
12 t = 0 : 1/fs : t_total - 1/fs;
13 Nsamples = length(t);
14
15 a = 0.8; % 调制指数
16
17 % 窄带噪声参数
18 N0 = 1e-4; % 双边带功率谱密度 (W/Hz)
19 Pn_c = N0 * B / 2; % 同相分量 n_c(t) 的功率
20
21 % 低通滤波器设计
22 f_cut = B / 2; % 低通截止频率 (Hz)
23 filter_order = 200; % 滤波器阶数
24 h_lp = fir1(filter_order, f_cut/(fs/2), 'low');
25
26 %% 生成窄带高斯噪声的同相分量和正交分量
27 n_c_filt = filter(h_lp, 1, randn(1, Nsamples));
28 actual_power_c = var(n_c_filt);
29 n_c = n_c_filt * sqrt(Pn_c / actual_power_c);
30
31 n_s_filt = filter(h_lp, 1, randn(1, Nsamples));
32 actual_power_s = var(n_s_filt);
33 n_s = n_s_filt * sqrt(Pn_c / actual_power_s);
34
35 % 载波
36 carrier_cos = cos(2*pi*fc*t);
37 carrier_sin = sin(2*pi*fc*t);
38
39 n_t = n_c .* carrier_cos - n_s .* carrier_sin;
40
41 %% 基带信号调制解调
42 bits = randi([0, 1], 1, L);
43 symbols = 2*bits - 1;

```

```

44 m = repelem(symbols, samples_per_symbol);
45
46 s_t = (1 + a * m) .* carrier_cos;
47
48 r_t = s_t + n_t;
49
50 demod = 2 * r_t .* carrier_cos;
51 demod_filt = filtfilt(h_lp, 1, demod);
52
53 % 滤除滤波器初始瞬态
54 start_idx = filter_order + 1;
55 t_adj = t(start_idx:end);
56 m_adj = m(start_idx:end);
57 n_c_adj = n_c(start_idx:end);
58 demod_out = demod_filt(start_idx:end);
59
60 demod_ac = demod_out - mean(demod_out);
61 am_theory_ac = a * m_adj;
62
63 %% 绘图
64 figure('Name', 'AM调制与解调实验', 'Position', [100, 100, 1200, 800]);
65
66 % 子图1: 窄带高斯噪声同相分量 n_c(t)
67 t_limit_noise = 0.03;
68 idx_noise = t_adj <= t_limit_noise;
69 subplot(2,3,1);
70 plot(t_adj(idx_noise), n_c_adj(idx_noise), 'b', 'LineWidth', 1);
71 xlabel('时间/(s)'); ylabel('幅度');
72 title('窄带高斯噪声同相分量 n_c(t)');
73 grid on;
74
75 % 子图2: 基带信号 m(t)
76 subplot(2,3,2);
77 plot(t, m, 'r', 'LineWidth', 1.2);
78 xlabel('时间/(s)'); ylabel('幅度');
79 title('矩形基带信号 m(t)');
80 grid on;
81 ylim([-1.2, 1.2]);
82
83 % 子图3: AM 同向分量 = 1 + a*m(t)
84 subplot(2,3,3);
85 am_envelope = 1 + a * m;
86 plot(t, am_envelope, 'm', 'LineWidth', 1.2);
87 xlabel('时间/(s)'); ylabel('幅度');
88 title('AM同向分量 1+a*m(t)');
89 grid on;
90 ylim([-0.2, 2.2]);
91
92 % 子图4: AM 已调信号 s(t)
93 t_carrier_limit = 5 / fc; % 5 个载波周期

```

```

94 idx_s = t <= t_carrier_limit;
95 subplot(2,3,4);
96 plot(t(idx_s), s_t(idx_s), 'g', 'LineWidth', 1);
97 xlabel('时间(s)'); ylabel('幅度');
98 title('AM已调信号 s(t)');
99 grid on;
100
101 % 子图5: AM 相干解调输出
102 subplot(2,3,5);
103 plot(t_adj, am_theory_ac, 'r--', 'LineWidth', 1.2); hold on;
104 plot(t_adj, demod_ac, 'b', 'LineWidth', 1);
105 xlabel('时间(s)'); ylabel('幅度');
106 title('AM相干解调输出 (去直流)');
107 legend('理论 a·m(t)', '解调输出 a·m(t)+n_c(t)', 'Location', 'best');
108 grid on;
109 ylim([-1.5, 1.5]);
110 hold off;
111
112 % 子图6: 留空
113 subplot(2,3,6);
114 axis off;
115
116 sgtitle('AM调制与解调实验波形', 'FontSize', 14);
117 tight;

```

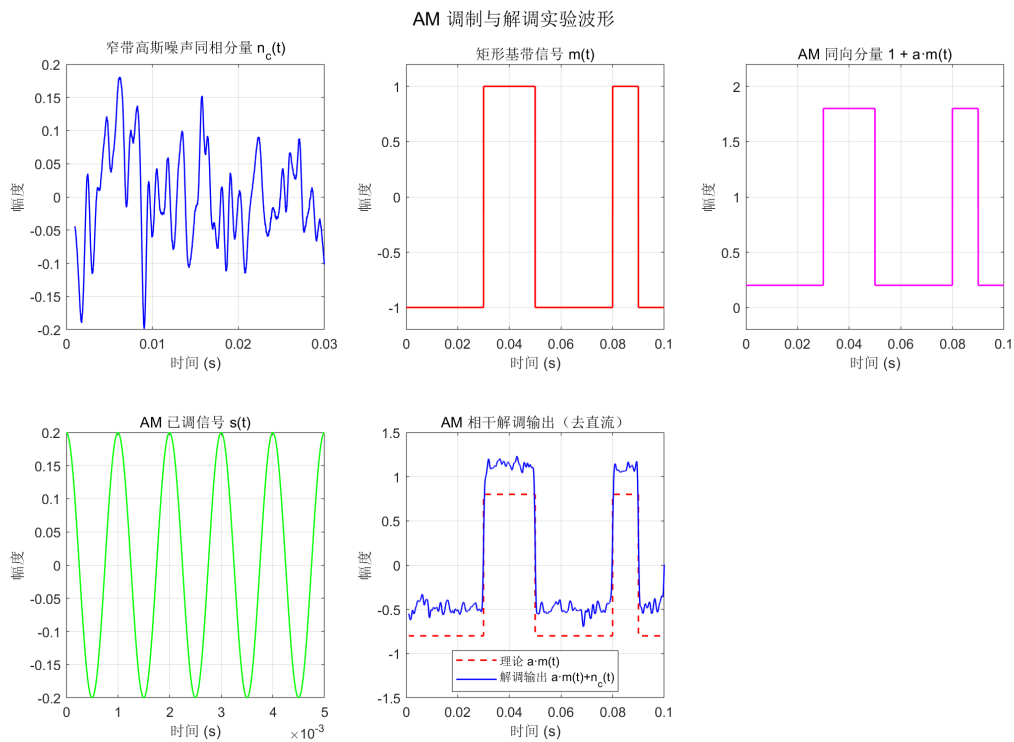


图 6: AM 调制解调波形